TRIBHUVAN UNIVERSITY
Institute of Engineering | BCT Program

OPERATING SYSTEM (CT 656)
CHAPTER 5: FILE SYSTEMS

Complete Exam Answer Guide | 6 Hours | 10 Marks

Topic No.	Syllabus Content
5.1	File: Name, Structure, Types, Access, Attribute, Operations
5.2	Directory and File Paths
5.3	File System Implementation
5.3.1	Selecting Block Size
5.3.2	Impact of Block Size Selection
5.3.3	Implementing File: Contiguous Allocation, Link List Allocation, Link List Allocation with Table, Inode
5.3.4	Implementing Directory
5.4	Impact of Allocation Policy on Fragmentation
5.5	Mapping File Blocks on The Disk Platter
5.6	File System Performance
5.7	Example File Systems: CD ROM file system, MS-DOS file system, Unix File system

Question 1: File Definition & File Implementation Methods

Q: Define File. Describe the implementation of a file system using linked lists and i-nodes.

Asked in: 2082 Kartik [1+6], 2081 Ashwin [4+6], 2078 Chaitra [2+6]

Marks: [1+6] = 7 marks

1.1 Definition of File

A file is a named collection of related information stored on secondary storage. From the user's perspective, a file is the smallest allotment of logical secondary storage. Files are managed by the OS through the file system.

File Attributes:

- Name: Human-readable symbolic identifier
- Identifier: Unique tag (inode number) identifying file within file system
- Type: Regular file, directory, symbolic link, device file, etc.
- Location: Pointer to device and address of file on device
- Size: Current file size in bytes or blocks
- Protection: Access control (read/write/execute permissions)
- Time, Date, User ID: For security and usage monitoring

1.2 Linked List Allocation (without Table)

Each file is stored as a linked list of disk blocks. Each block contains:

- Data portion
- Pointer to the next block (last block has null pointer)

Advantage	Disadvantage	Notes
No external fragmentation	Random access is slow (must traverse list)	Pointers waste space
No wasted space for large files	Pointer lost = data lost (unreliable)	No contiguous blocks needed

1.3 Linked List with Table (FAT — File Allocation Table)

The linked list pointers are stored in a separate table (FAT) in memory, not in the data blocks themselves. FAT entry for each disk block holds the next block number (or EOF/NULL).

Advantage	Disadvantage	Example
Random access possible (traverse table in RAM)	Entire FAT must be in memory	MS-DOS FAT16, FAT32
Blocks fully available for data	Large FAT for large disks	Windows file system

1.4 I-node (Index Node) — Used in UNIX

An inode is a data structure that stores all metadata about a file AND pointers to its data blocks. Each file has exactly one inode.

Inode Structure:

- File mode (type + permissions)
- Owner UID and GID
- File size in bytes
- Timestamps (created, modified, accessed)
- Direct block pointers (first 10-12 block addresses stored directly)
- Single indirect pointer: Points to a block of block addresses
- Double indirect pointer: Points to block of single-indirect block addresses
- Triple indirect pointer: For very large files

Advantage	Disadvantage	Max File Size
Fast random access	Fixed number of inodes at format time	Very large files supported
Inode loaded only when file is open	Small files waste inode space	Unix/Linux standard

1.5 Contiguous Allocation

File stored in consecutive disk blocks. Directory stores only start block and length.

- Advantage: Excellent read performance, simple implementation
- Disadvantage: External fragmentation, file growth difficult
- Best for: Read-only file systems like CD-ROMs

Question 2: Inode Calculation — Memory Space for FAT

Q: What is Inode? Why is it superior to other file allocation approaches? Consider a 20-GB disk with 8-KB block size. How much memory will be occupied if contiguous allocation is used for file allocation? Assume that each FAT entry takes 4 bytes.

Asked in: 2080 Ashwin [5]

Marks: [5] marks

2.1 Why Inode is Superior

Inode is superior to other file allocation methods because:

- 1. Fast Random Access: Any block can be reached directly through direct/indirect pointers, unlike linked list which needs sequential traversal
- 2. Efficient Memory Use: Only inode of open files loaded into memory (unlike FAT which loads entire table)
- 3. Supports Large Files: Triple indirection allows files many GB in size
- 4. Rich Metadata: Stores permissions, ownership, timestamps in one place
- 5. No External Fragmentation: Blocks need not be contiguous

2.2 FAT Memory Calculation

Given: 20 GB disk, 8 KB block size, 4 bytes per FAT entry

```
Total disk size    = 20 GB = 20 × 1024 × 1024 × 1024 bytes = 21,474,836,480 bytes
Block size         = 8 KB = 8 × 1024 = 8,192 bytes
Number of blocks   = 21,474,836,480 / 8,192 = 2,621,440 blocks
FAT entry size     = 4 bytes
Total FAT size     = 2,621,440 × 4 bytes = 10,485,760 bytes = 10 MB
```

The FAT table would occupy 10 MB of memory — this must always be kept in RAM for the file system to work, making it impractical for very large disks.

This is why inode-based systems (Unix) are preferred: only inodes of OPEN files are in memory.

Question 3: File Allocation Methods — Comprehensive

Q: What is file attribute? List the File System performance indicator. Explain the file allocation methods with its advantages and disadvantages. Also suggest which file organization technique is most appropriate for 'tape storage' and why?

Asked in: 2079 Chaitra [1+2+6+1], 2079 Jestha [1+2+6+1], 2080 Ashwin [1+2+6+1]

3.1 File Attributes

File attributes (metadata) include: Name, Type, Identifier, Location (device + address), Size (bytes/blocks/records), Protection (access control list), Time/Date/User ID for auditing.

3.2 File System Performance Indicators

- Access Time: Time to locate and access a file
- Throughput: Data transfer rate (bytes/second)
- Storage Utilization: Percentage of disk used effectively
- Reliability: Data integrity and recovery capability
- Scalability: Performance as disk size/number of files grows

3.3 Comprehensive Comparison of File Allocation Methods

Feature	Contiguous	Linked List / FAT	Indexed (Inode)
Storage	Consecutive blocks	Scattered blocks with pointers	Scattered blocks with index
Directory Entry	Start block + length	Start block	Inode number
Sequential Access	Excellent	Good	Good
Random Access	Excellent (direct calc)	Poor (FAT) / Very slow (linked)	Excellent
External Frag.	Yes (major problem)	None	None
Internal Frag.	Possible last block	None	None
File Growth	Difficult	Easy	Easy
Best For	CD-ROM, read-only	Small/sequential files	General purpose (Unix)
Example	CD-ROM FS	MS-DOS (FAT)	Unix/Linux (ext4)

3.4 Best Method for Tape Storage

Contiguous Allocation is most appropriate for tape storage.

Reasons:

- Tape is a sequential access medium — random access is not possible
- Contiguous allocation stores all file blocks together, enabling fast sequential reads
- No need to follow pointers across different parts of tape
- External fragmentation is not a concern for sequential media like tape

Question 4: Directory Implementation

Q: What is meant by file attributes? Discuss any one technique of implementing directories in detail.

- **Asked in: 2080 Ashwin [5], 2079 Chaitra [5]**
- **Marks: [5] marks**

4.1 Directory Structure Techniques

Technique	Description	Example
Linear List	Fixed-size entries: filename + metadata	Simple but slow search O(n)
Hash Table	Hash filename → index in table	Fast O(1) lookup but collisions
Tree (B-tree)	Sorted tree of filenames	Used in HFS+, NTFS, ext4
Single-level Dir.	One global directory for all files	Simple, but naming conflicts
Two-level Dir.	Separate directory per user	No conflicts across users
Hierarchical	Tree structure (Unix-like)	Most common today

4.2 Hash Table Directory Implementation — Detail

A hash table is used to quickly locate directory entries:

- A hash function maps file name → index in a fixed-size table
- Each table entry points to a linked list of directory entries (to handle collisions)
- Search: Hash the name → check that table slot → traverse chain if needed

```
Entry: [hash_index] → {filename, inode_no, next_entry_ptr}
```

Advantages: O(1) average search time, works well for large directories.

Disadvantages: Fixed hash table size, hash collisions require chaining.

Question 5: Free Disk Space Management

Q: Describe different file allocation methods. Explain free disk space management with example.

- **Asked in: 2079 Ashwin [6], 2079 Jestha [6]**
- **Marks: [6] marks**

5.1 Free Space Management Methods

Method	How It Works	Pros / Cons
Bit Vector (Bitmap)	One bit per block: 0=free, 1=used	Easy to find contiguous blocks / must be in memory
Free List (Linked List)	Linked list of all free blocks	Low overhead / not efficient for contiguous space
Grouping	First free block stores n free block addresses + 1 pointer to next group	Efficient for large number of free blocks
Counting	Stores (first_free_block, count) pairs	Efficient when many consecutive free blocks

5.2 Bitmap (Bit Vector) Example

Assume 16 disk blocks. Status: blocks 0,1,4,5,8,9 free; 2,3,6,7,10,11,12,13,14,15 used:

```
Bit vector: 1 1 0 0 1 1 0 0 1 1 0 0 0 0 0 0
            0 1 2 3 4 5 6 7 8 9 ...
```

Bit=1 means free, Bit=0 means used (or vice versa depending on implementation).

Contiguous free region: Blocks 4,5 (2 consecutive) or blocks 8,9 (2 consecutive).

Question 6: RAID — Disk Array Concepts

Q: Describe File System for operating system. Why are output files for the printer normally spooled on disk before being printed? Explain how RAID level 1 does differ from RAID level 0.

- **Asked in: 2080 Chaitra [4+2+2]**
- **Marks: [4+2+2] = 8 marks**

6.1 File System Structure

A file system provides the mechanism for storage, retrieval, and naming of files. It has a layered structure:

- Application programs → File operations (create, read, write, delete)
- Logical File System → Manages metadata (directory structure, file table)
- File Organization Module → Knows about files, logical blocks, physical blocks
- Basic File System → Sends generic commands to device driver
- I/O Control → Device drivers and interrupt handlers
- Devices → Disk drives, optical drives

6.2 Why Printer Output is Spooled on Disk

Spooling (Simultaneous Peripheral Operations On-Line) stores printer output temporarily on disk because:

- Printer is slow — process would stall waiting for each character to print
- Multiple jobs can be queued — print spooler schedules them
- Process completes immediately after writing to spool — freed for other work
- Printer errors don't affect the application process

Example: User prints 100 pages. File written to /var/spool/cups in seconds. Printer takes 10 minutes. Process is free immediately.

6.3 RAID Level 0 vs RAID Level 1

Feature	RAID 0 (Striping)	RAID 1 (Mirroring)
Mechanism	Data striped across multiple disks	Exact copy on second disk
Redundancy	None — no fault tolerance	Full redundancy
Read Performance	Excellent (parallel reads)	Excellent (read from either)
Write Performance	Excellent	Moderate (must write twice)
Disk Overhead	0% (100% capacity used)	50% (capacity halved)
Fault Tolerance	None — 1 disk fails = data lost	Yes — survives single disk failure
Use Case	High performance, scratch data	Critical data, databases

Question 7: File System Examples — Unix, MS-DOS, CD-ROM

Q: What is the role of file system? How can files be implemented? List any three of them with advantages and disadvantages of each.

- **Asked in: 2078 Chaitra [2+6], 2079 Chaitra [2+3], 2081 Ashwin [4+6]**
- **Marks: [2+6] = 8 marks**

7.1 Role of File System

- Provides abstraction: Programs use filenames, not disk addresses
- Manages free space and allocates disk blocks
- Maintains directory hierarchy for organization
- Enforces access control and permissions
- Ensures data integrity and recovery

7.2 Unix File System (UFS/ext)

Structure: Boot block | Super block | Inode table | Data blocks

- Uses inodes for file metadata and block pointers
- Hierarchical directory tree rooted at '/'
- Everything is a file: devices, pipes, sockets treated as files
- Hard links and symbolic links supported

Advantage: Fast, reliable, supports large files. Disadvantage: Fixed inode count.

7.3 MS-DOS File System (FAT)

Structure: Boot sector | FAT1 | FAT2 (backup) | Root directory | Data area

- Uses File Allocation Table (linked list with table) for block management
- 8.3 filename format originally (now supports long names via VFAT)
- FAT12, FAT16, FAT32 differ in number of bits for cluster address

Advantage: Simple, widely compatible. Disadvantage: No permissions, FAT in memory.

7.4 CD-ROM File System (ISO 9660)

Structure: System area | Volume descriptor | Path table | Directory records | File data

- Uses contiguous allocation — all files stored in consecutive blocks
- Read-only medium — no need for free space management
- Fixed directory structure written at creation time

Advantage: Maximum read performance. Disadvantage: Cannot be modified after writing.

Question 8: Short Notes — File System Concepts

Q: Write short notes on: (a) File System Layout (b) Inode vs FAT (c) File System Performance

- **Asked in: Multiple years [3×3]**
- **Marks: [3×3] = 9 marks**

8.1 File System Layout (Disk Structure)

Region	Content
MBR	Master Boot Record — partition table, bootloader code (sector 0)
Boot Block	OS kernel bootstrap loader — loaded to memory at boot
Superblock	File system info: block size, total blocks, free count, inode count
Inode Table	Array of all inodes (Unix) or FAT table (MS-DOS)
Data Blocks	Actual file content, directory entries

8.2 File System Performance Techniques

- Block Caching: Most recently used disk blocks cached in RAM (buffer cache)
- Read-Ahead: OS reads next blocks before they're requested (sequential access)
- Disk Layout Optimization: Store related blocks near each other (cylinder groups)
- Journaling: Write-ahead log prevents corruption on crash (ext4, NTFS)
- SSD Optimization: TRIM command frees blocks, wear leveling extends SSD life